

Agentic AI Engineer Capstone

Transaction Dispute & Fraud Triage Copilot — Multi-Agent Dispute Handling

Business Case AAIE_AGT_001_BFS · Domain: Banking & Finance — Cards & Payments Disputes · Track: Agentic AI Core + Context Engineering & Memory + MCP & Interoperability

1. Project Identity

Business Case Title	Transaction Dispute & Fraud Triage Copilot — Multi-Agent Dispute Handling
Business Case ID	AAIE_AGT_001_BFS
Domain	Banking & Finance — Cards & Payments Disputes
Project Type	Agentic AI Capstone — Agentic AI Core + Context Engineering & Memory + MCP & Interoperability
Cohort	(filled by Operations team)

2. Engagement Overview

Duration	15 hours
Format	Individual
Evaluation Mode	Automated static review of the submitted Git repository against the Agentic AI Rubric (22 parameters / 100 marks across 7 categories). Rubric parameters scored with Google Gemini (not Claude); presence and numeric-threshold parameters scored deterministically in Python.
Submission	Push final code to your assigned Virtusa GitLab repository by the cohort cut-off.
Review Output	Per-learner Excel report (Summary, Categories, Scorecard, Detailed, Improvement).
Grade Bands	Distinction ≥ 90 · Merit 75-89 · Pass 60-74 · Not Yet Passed < 60

What is evaluated: agent architecture on LangGraph (typed state, graph topology, conditional routing, checkpointing, structured output), agent patterns and multi-agent orchestration, context engineering, tiered memory with verified cross-session persistence, a custom MCP server consumed by the agent, and an agentic-RAG tool.

What is not evaluated: the visual polish of any interface, the sophistication of the domain heuristics, or which optional comparison framework you choose. You are judged on agentic engineering and committed evidence.

3. Problem Statement & Expected Solution

3.1 Problem

A bank's dispute-operations team needs to triage transaction disputes faster. When a customer raises a disputed or suspicious transaction, a copilot should capture it, gather transaction and customer context, surface fraud signals, check chargeback eligibility against the rules, and draft a resolution for a human agent — carrying context across a multi-turn, potentially multi-session case. You build this as a LangGraph multi-agent system with a custom MCP tool server, engineered context, and tiered memory.

3.2 Your Role

Agentic AI Engineer. You design the agent graph and state, choose and justify single-vs-multi-agent orchestration, build a custom MCP server and wire it into the agent, engineer context (write / select / compress / isolate), and implement a tiered memory that survives across sessions — all evidenced by committed traces and tests.

3.3 Expected Solution

A working multi-agent application delivered as a Git repository that demonstrates:

- A LangGraph graph with a typed state object, a supervisor routing an incoming transaction dispute to specialized worker agents (intake, fraud-signal, chargeback-eligibility, resolution-draft), and conditional edges driven by state.
- A custom MCP server exposing at least 2 tools and 1 resource, consumed by the agent via langchain-mcp.

/0/#search/agentica+ai+/FMfcgzQhWLKPNbCgthvLMLmjkZpDsTsz?projector=1&messagePartId=0.1

- A custom MCP server exposing at least 2 tools and 1 resource, consumed by the agent via langchain-mcp-adapters, with a committed run transcript showing tool invocation.
- Engineered context (write / select / compress / isolate) with summarization middleware and quarantine of untrusted customer complaint text.
- A tiered memory layer whose cross-session persistence is proven by a committed test and its output log, plus an eviction / importance policy.
- An agentic-RAG tool the agent decides to call for dispute-rule and chargeback-reason lookups, and a reflection / self-healing loop with an evidenced trace.

3.4 Applicable Rules

- **Synthetic-Data Rule.** Use only synthetic / dummy data you generate yourself — no real customer, account, or card (PAN) data, and no Virtusa confidential data.
- **Evidence-in-Repo Rule.** Only committed artifacts are scored. Run transcripts, tool-call logs, the memory-persistence test and its output, and traces must be committed — uncommitted behavior does not count.
- **Reproducibility Rule.** The system must run from a single documented command with committed sample dispute inputs and a README quick-start.
- **AC-Traceability Rule.** Each Acceptance Criterion must be referenced by at least one test or committed evidence artifact carrying its AC-NN identifier.
- **Context-Isolation Rule.** Free-text customer complaint content is untrusted; it must be isolated (quarantined) and never treated as instructions to the agent.
- **Open-Source & No-Docker Rule.** Use only the approved open-source stack with Google Gemini as the LLM provider. The project must build, run, and be evaluated with pip + Python alone — no Docker and no external database service.

4. Technology & Framework Stack

The stack is fixed to an open-source toolchain with Google Gemini as the only model provider. The project must build, run, and be evaluated with pip + Python alone — no Docker or external database service.

Layer	Approved tool (open source unless noted)
Language	Python 3.11+
Agent Framework	LangGraph (MIT, required); CrewAI optional for the single-vs-multi comparison
LLM Provider	Google Gemini (the only approved model provider; not Claude)
Interoperability	langchain-mcp-adapters (MIT)
Memory	semantic memory; Chroma / FAISS for

	Comparison
LLM Provider	Google Gemini (API) — the only approved model provider; not Claude
Interoperability	MCP Python SDK server (stdio) + langchain-mcp-adapters (MIT)
Memory	langgraph-checkpoint-sqlite (SQLite file) + LangMem; Chroma / FAISS for semantic memory
Embeddings	Sentence-Transformers (local, open source)

Layer	Approved tool (open source unless noted)
Retrieval (tool)	Chroma or FAISS for the agentic-RAG lookup tool
Interface (optional)	CLI · Streamlit · Gradio · FastAPI

5. Acceptance Criteria & Non-Functional Requirements

5.1 Functional Acceptance Criteria

Note. Each AC must have at least one test or committed evidence artifact referencing its AC-NN identifier.

ID	Criterion
AC-01	The system is built on LangGraph with an explicit typed state object (TypedDict / Pydantic) shared across nodes.
AC-02	A supervisor / orchestrator routes an incoming transaction dispute to specialized worker agents (intake, fraud-signal, chargeback-eligibility, resolution-draft).
AC-03	The graph uses conditional edges to route on state (e.g., escalate suspected fraud to manual review; short-circuit disputes outside the chargeback window).
AC-04	Node / agent outputs are validated structured objects (Pydantic) at handoff boundaries.
AC-05	A checkpointer persists graph state so a dispute case can be paused and resumed.
AC-06	The copilot maintains tiered memory (short-term working + long-term / semantic) and recalls a fact from an earlier turn.
AC-07	Memory persists across sessions; a committed test starts a new session and shows recall of prior-session facts; its output log is committed.
AC-08	A memory eviction / importance policy (TTL, LRU, or importance-weighted) is implemented and documented.
AC-09	A custom MCP server exposes ≥ 2 tools and 1 resource relevant to the domain (e.g., transaction, dispute, customer, profile, fraud, rules, test, dispute, handling, request, resource).

m/mail/u/0/#search/agentica+ai+/FMfcgzQhWLKPNbCgthvLMLmjkZpDsTsz?projector=1&messagePartId=0.1

AC-09	A custom MCP server exposes ≥ 2 tools and 1 resource relevant to the domain (e.g., transaction_lookup + customer_profile + fraud_rules tools; dispute_handling_manual resource).
AC-10	The agent consumes the MCP server via langchain-mcp-adapters; a committed transcript shows the agent invoking an MCP tool.
AC-11	An agentic-RAG tool is available and the agent decides when to call it for dispute-rule and chargeback-reason lookups (retrieval inside the loop, not a fixed step).
AC-12	The system implements a reflection or self-healing / fallback loop (e.g., re-plan on tool failure or low-confidence output) with an evidenced trace.

5.2 Non-Functional Requirements

ID	Requirement
NFR-01	No secrets or API keys committed; env-var configuration with a committed .env.example.
NFR-02	System runs end-to-end from a single documented command with committed sample dispute inputs and a README quick-start.
NFR-03	Untrusted free-text customer complaint content is isolated (context quarantine) and never trusted as instructions.
NFR-04	Structured JSON logs / traces of agent runs are committed as evidence.
NFR-05	All data is synthetic; any PII is synthetic and never written to logs in plaintext.
NFR-06	The single-vs-multi-agent decision and the framework choice are documented with rationale.
NFR-07	Graceful degradation on tool / model failure: timeouts, retries, and explicit exit conditions.
NFR-08	Context-window management: summarization / compression is applied for long threads.

6. Functional Scope

6.1 In Scope

- A LangGraph multi-agent graph: typed state, supervisor + worker agents, conditional routing, checkpointing, structured output.

- A custom MCP server (≥ 2 tools + 1 resource) integrated into the agent and exercised in a committed

- A custom MCP server (≥ 2 tools + 1 resource) integrated into the agent and exercised in a committed transcript.
- Context engineering (write / select / compress / isolate), summarization middleware, and quarantine of untrusted customer complaint text.
- Tiered memory with verified cross-session persistence and an eviction / importance policy.
- An agentic-RAG dispute-rule and chargeback-reason lookups tool and a reflection / self-healing loop.
- A minimal interface (CLI / Streamlit / Gradio / FastAPI) to drive an incoming transaction dispute through the graph.

6.2 Out of Scope

- Real core-banking / card-network connectivity or actual chargeback submission.
- Real customer or confidential data of any kind.
- Production deployment, multi-region, authentication, and real back-end system integration.
- Front-end visual polish and any real transaction / action execution — heuristics and stubs are sufficient.

7. Implementation Expectations

The rubric scores committed evidence. Each category below states what must exist in the repository. Marks per category are shown in Section 9.

7.1 Business & Requirements (10 marks)

- docs/business-case.md covering problem, actors, and success metrics for the transaction-dispute triage workflow.
- Acceptance criteria in testable form (AC-NN).
- A documented single-vs-multi-agent decision and framework-choice rationale.

7.2 Agent Architecture & LangGraph (24 marks)

- A typed state object (TypedDict / Pydantic) shared across nodes.
- A graph topology with a supervisor and specialized worker agents (nodes + edges).
- Conditional routing / decision edges driven by state.
- A configured checkpointer for state persistence at node boundaries.

7.3 Patterns & Multi-Agent (10 marks)

- A configured checkpointer for pause / resume; structured output validated at node boundaries.

7.3 Patterns & Multi-Agent (18 marks)

- An implemented agent pattern (ReAct / plan-execute / reflection) with a short rationale.
- Working multi-agent orchestration (supervisor or swarm) evidenced by a committed run transcript.
- A reflection or self-healing / fallback loop with an evidenced trace.

7.4 Context Engineering (12 marks)

- Explicit write / select / compress / isolate strategies mapped in a short doc.
- Summarization / compression middleware for long threads.
- Context quarantine isolating untrusted customer complaint text.

7.5 Memory Systems (14 marks)

- Tiered memory (short-term working + long-term / semantic).
- Cross-session persistence proven by a committed test and its output log.

- An eviction / importance policy (TTL / LRU / importance-weighted).

7.6 MCP & Interoperability (14 marks)

- A custom MCP server exposing ≥ 2 tools and 1 resource.
- Integration via langchain-mcp-adapters with a committed tool-call transcript.
- An integration-decision writeup (MCP vs API vs direct-DB vs A2A).

7.7 Agentic RAG & Reproducibility (8 marks)

- An agentic-RAG tool the agent can use with a committed transcript and a feedback-reason lookups.
- Engineering hygiene: README quick-start, single-command run, no secrets, committed traces.

7.7 Agentic RAG & Reproducibility (8 marks)

- An agentic-RAG tool the agent calls on demand for dispute-rule and chargeback-reason lookups.
- Engineering hygiene: README quick-start, single-command run, no secrets, committed traces.

8. Expected Outcomes & Deliverables

By the end of 15 hours, the submitted Git repository must contain the Mandatory items below. Good-to-Have items differentiate Merit and Distinction submissions.

8.1 Mandatory

- Working multi-agent system runnable locally via a single command, with committed sample dispute inputs and README quick-start.
- docs/business-case.md and specs with AC-NN acceptance criteria.
- LangGraph graph: typed state, supervisor + workers, conditional routing, checkpointing, structured output.
- Custom MCP server (≥ 2 tools + 1 resource) + adapter integration + committed tool-call transcript.
- Context engineering (write/select/compress/isolate) + summarization middleware + quarantine of untrusted customer complaint text.
- Tiered memory + committed cross-session persistence test and log + eviction policy.
- Agentic-RAG tool + reflection/self-healing loop with trace; .env.example; no committed secrets.
- PR-driven Git history: at least 3 PR-driven merges (git merge --no-ff); no direct pushes to main.

8.2 Good-to-Have

- A single-vs-multi-agent comparison run (supervisor variant vs single-agent variant) with observations.
- A second MCP server (filesystem / database) or an A2A demonstration.
- Importance-weighted memory with semantic recall over Chroma / FAISS / LangGraph Store.
- A fraud-score-aware routing variant that escalates high-risk disputes to a dedicated review path.
- A lightweight UI showing the graph's routing decisions and memory state.

9. Evaluation Rubric

22 parameters across 7 categories, 100 marks. Scoring is model-portable (Google Gemini — not Claude) and reads only committed repository evidence. Presence / threshold parameters (marked Deterministic) are scored in Python with no model judgment.

om/mail/u/0/#search/agentic+ai+/FMfcgzQhWLKPNbCgthvLMLmjkZpDsTsz?projector=1&messagePartId=0.1

agentic ai

22 parameters across 7 categories, 100 marks. Scoring is model-portable (Google Gemini — not Claude) and reads only committed repository evidence. Presence / threshold parameters (marked Deterministic) are scored in Python with no model judgment.

Category	Parameters (max)	Marks
Business & Requirements	Business-case clarity (4) · AC definition (3) · Single-vs-multi justification (3)	10
Agent Architecture & LangGraph	Typed state (5) · Graph topology (6) · Conditional routing (6) · Checkpointing — Deterministic (4) · Structured output (3)	24
Patterns & Multi-Agent	Pattern implementation (6) · Multi-agent orchestration + transcript (7) · Reflection / self-healing (5)	18

Category	Parameters (max)	Marks
Context Engineering	Write/select/compress/isolate (5) · Compression middleware (4) · Context quarantine (3)	12
Memory Systems	Tiered memory (5) · Cross-session persistence — Deterministic (6) · Eviction policy (3)	14
MCP & Interoperability	Custom MCP server — Deterministic (6) · Adapter integration + tool-call log (5) · Integration decision (3)	14
Agentic RAG & Reproducibility	Agentic-RAG tool (4) · Reproducibility / secrets (4)	8
TOTAL	22 parameters	100

Grade Band	Range
Distinction	≥ 90
Merit	75 – 89
Pass	60 – 74
Not Yet Passed	< 60

On completion you will have demonstrated a transaction-dispute and fraud-triage copilot — a stateful, multi-agent copilot with LangGraph, custom tooling, engineered context, and

Category	Parameters (max)	Marks
Context Engineering	Write/select/compress/isolate (5) - Compression middleware (4) - Context quarantine (3)	12
Memory Systems	Tiered memory (5) - Cross-session persistence — Deterministic (6) - Eviction policy (3)	14
MCP & Interoperability	Custom MCP server — Deterministic (6) - Adapter integration + tool-call log (5) - Integration decision (3)	14
Agentic RAG & Reproducibility	Agentic RAG tool (4) - Reproducibility / secrets (4)	8
TOTAL	22 parameters	100

Grade Band	Range
Distinction	≥ 90
Merit	75 - 89
Pass	60 - 74
Did Not Pass	< 60

On completion you will have demonstrated the skill industry is seeking: building a transaction-dispute and fraud-triage copilot — a stateful, multi-agent copilot with LangGraph, custom MCP tooling, engineered context, and memory that survives across sessions — with evidence, not just a demo that worked once.

I